22. Sorting and Searching

22. Sorting and Searching

Phase: Phase 4 — Algorithms and data structures
Estimated time: 1–2 days
Slug: `22-sorting-searching`

Learning objectives

- Use sort package effectively
- Implement binary search correctly
- Know comparison contracts
- Avoid off-by-one errors

Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

Why this matters

Sorting and searching are building blocks for indexes, schedulers, and caches. Go's sort package and binary search repay fluency.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

Core concepts

- sort.Ints / sort.Slice with less functions.
- Binary search needs sorted order — verify invariants.
- stable vs unstable sorts matter for tie-breaking.
- search returns index or insertion point patterns.
- Custom order via sort.Interface or Slice.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-sorting && cd /tmp/gw-sorting
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```
import (
    "fmt"
    "sort"
)

func main() {
    xs := []int{5, 1, 3}
    sort.Ints(xs)
    i := sort.SearchInts(xs, 3)
    fmt.Println(xs, i)
}
```

Run it:

```
go run .
```

Expected output:

```
[1 3 5] 1
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

Mini-lab (`go run` steps)

1. Sort structs by a field with sort.Slice.
2. Binary search for a target; handle not-found.
3. Compare stable vs unstable with equal keys.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

Curriculum notes (expanded from the compact draft)

sort package

```
sort.Ints(xs)
sort.Slice(people, func(i, j int) bool { return people[i].Age < people[j].Age })
```

Binary search

Requires sorted input. Classic bug: midpoint overflow (use `lo+(hi-lo)/2`).

```
func Search(xs []int, target int) int {
    lo, hi := 0, len(xs)
    for lo < hi {
        mid := lo + (hi-lo)/2
        if xs[mid] < target { lo = mid + 1 } else { hi = mid }
    }
    if lo < len(xs) && xs[lo] == target { return lo }
    return -1
}
```

Stability

`sort.SliceStable` preserves equal-element order — useful for multi-key sorts.

Failure modes to watch while coding

- Binary search on unsorted data
- Inclusive vs exclusive bounds confusion
- Expensive comparisons in comparators

When a test fails, ask which failure mode you hit before changing code at random.

Idioms vs Python pitfalls

- list.sort vs sorted; Go sorts in place commonly.
- bisect module ≈ sort.Search.
- key= functions ≈ less closures in sort.Slice.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

Going deeper

When data is huge, consider whether you need a full sort or a heap/selection.

Practice checklist

Before the exercise, confirm you can:

1. Explain "Sorting and Searching" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

Common mistakes (especially from Python instincts)

- Binary search on unsorted data
- Inclusive vs exclusive bounds confusion
- Expensive comparisons in comparators

Exercises

Work in `exercises/22-sorting-searching/`. Read that folder's `README.md` before you code.

What you will build

Implement search/sort helpers to match exercise tests.

Study loop

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

Do not open `solutions/22-sorting-searching/` until you have an honest failing attempt.

Self-check

Answer without scrolling up:

1. Preconditions for binary search?
2. When use SliceStable?
3. What is mid overflow?

Further reading

- https://pkg.go.dev/sort